

claude-windows / a9fd4426-f234-4e4a-bfb4-1f2c8255c661

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-muneem\a9fd4426-f234-4e4a-bfb4-1f2c8255c661\subagents\workflows\wf_076beba8-59c\agent-a4e25702f7000db3a.jsonl`  
- SHA-256: `99ad44d7bcd59624d867251acb244da489f60f1702b7b59300e170c220657895`  
- Source modified: `2026-05-31T04:18:40+00:00`  
- Imported at: `2026-07-05T16:48:20+00:00`  
- project: `wf_076beba8-59c`  
- session_id: `a9fd4426-f234-4e4a-bfb4-1f2c8255c661`
```

Transcript

1. user (2026-05-31T04:17:40.134Z)

Source Extractor

Research question: "Identify the best LOCAL (self-hosted, single NVIDIA CUDA GPU) vision / document-AI / OCR models ? and, as a clearly-flagged approval-gated fallback only, paid cloud API services ? for parsing BANK STATEMENT PDFs into structured transaction tables (date, narration/description, withdrawal/debit, deposit/credit, running balance), for a LOCAL-FIRST personal-finance tool ("muneem").

Target documents: Indian bank statements (HDFC, Axis, ICICI, AU Small Finance) ? dense, multi-column, often SEMI-RULED or borderless tables, sometimes multiple accounts per statement; both born-digital (text layer present) and the harder cases where pdfplumber's extract_tables/coordinate clustering is unreliable (e.g. Axis, whose column layout varies between statements).

HARD CONSTRAINTS (a model that violates these is unusable for us):

1. MUST run 100% LOCALLY for statement contents (highly sensitive). NO cloud APIs by default. Paid/cloud is acceptable ONLY as an explicitly-approved fallback ? still report the best cloud options but label them clearly as "cloud/sensitive ? approval-gated, not default".
2. MUST be usable from Python (transformers / vLLM / Ollama / ONNX Runtime / native lib).
3. License MUST be permissive & commercially safe for BOTH the CODE and the MODEL WEIGHTS: MIT, BSD, Apache-2.0, ISC, HPND. AVOID and explicitly FLAG anything GPL/LGPL/AGPL, CC-BY-NC / non-commercial, research-only, or custom "community"/acceptable-use licenses with restrictions. This is critical ? many document-AI models and their training datasets are non-commercial (e.g. LayoutLMv3 weights are CC-BY-NC; Surya and Marker use GPL-or-commercial dual licensing; several VLM weights ship custom restrictive licenses). Verify the WEIGHTS license, not just the repo license.
4. Hardware: a single consumer/prosumer NVIDIA GPU. Give VRAM tiers (~8-12 GB consumer, ~16-24 GB) and which models/quantizations fit each.

Evaluate and compare these LOCAL approaches ? for each give: code license, MODEL-WEIGHTS license (state explicitly), latest release / maintenance status as of 2025-2026, VRAM need + quantization options, Python integration path, and reliability specifically on DENSE FINANCIAL TABLES / multi-column statements:

- Open vision-language models (VLMs) for document/table understanding: Qwen2-VL and Qwen2.5-VL (note per-size license ? which sizes are Apache-2.0 vs restricted), MiniCPM-V 2.6, InternVL2 / 2.5, Microsoft Phi-3.5-vision, Mistral Pixtral, Meta Llama-3.2-Vision (flag the Llama Community License restrictions), Moondream2, GOT-OCR2.0, olmOCR and other OCR-specialized VLMs.
- OCR-specialist / document-AI pipelines: PaddleOCR + PP-Structure (table recognition), docTR, EasyOCR, Tesseract (already used), Microsoft Table Transformer / TATR (check code AND PubTables-1M dataset/weights license), Donut, Surya (FLAG license), Marker (FLAG license), MinerU / PDF-Extract-Kit, RapidOCR.
- Table-structure recognition specifically (cell/grid reconstruction for borderless tables): TATR, PP-Structure, UniTable, etc.

Also assess: for SEMI-RULED / borderless bank tables, is a prompt-driven VLM (page image -> JSON

transactions) more reliable than a classic OCR + table-structure pipeline? What are the accuracy/hallucination risks of VLMs on financial NUMBERS (transposed digits, dropped/duplicated rows), and how should output be VERIFIED? (muneem already machine-checks parses by balance reconciliation ? opening + sum(deposits) - sum(withdrawals) == closing, plus per-row running-balance walks and cross-checks against the bank's printed summary ? so any model's output can be automatically validated, which should inform the recommendation.)

Finally, briefly cover the best PAID/CLOUD fallbacks (clearly flagged approval-gated for sensitive data): AWS Textract (AnalyzeExpense / Tables), Google Document AI, Azure AI Document Intelligence, Reducto, LlamaParse, Anthropic/OpenAI vision ? note any self-hosting option and data-handling terms.

Deliver a RANKED shortlist of the most promising LOCAL options for muneem (NVIDIA GPU, Python, commercial-license-safe), each with license (code + weights), VRAM tier, maturity, strengths/weaknesses for dense bank-statement tables, and a concrete recommendation: which local model(s) to prototype for muneem's statement parsing and how they'd combine with the existing deterministic parsers + balance-reconciliation verification (e.g. VLM as a fallback/cross-check when deterministic parsing fails to reconcile). Cite sources and call out any license uncertainty explicitly."

Fetch and extract key claims from this source:

****URL:**** <https://github.com/PaddlePaddle/PaddleOCR/blob/main/docs/version3.x/algorithm/PP-StructureV3/PP-StructureV3.en.md>

****Title:**** PaddleOCR ? PP-StructureV3 algorithm docs (layout + table recognition ? Markdown/JSON)

****Found via:**** OCR + table-structure recognition pipelines (practitioner/implementation) search

Task

1. Use WebFetch to retrieve the page content.
2. Assess source quality: primary research/institution? secondary reporting? blog/opinion? forum? unreliable?
3. Extract 2-5 FALSIFIABLE claims that bear on the research question. Each claim must:
 - be a concrete, checkable statement (not vague generalities)
 - include a direct quote from the source as support
 - be rated central/supporting/tangential to the research question
4. Note publish date if available.

If the fetch fails or the page is irrelevant/paywalled, return claims: [] and sourceQuality: "unreliable".

Structured output only.

2. assistant (2026-05-31T04:17:43.183Z)

I'll fetch the PaddleOCR PP-StructureV3 documentation page and extract relevant claims.

3. assistant (2026-05-31T04:17:46.671Z)

I'll fetch the PaddleOCR PP-StructureV3 documentation page.

4. user (2026-05-31T04:18:01.113Z)

PP-StructureV3 Factual Claims for Bank Statement Table Parsing

(1) Core Capabilities

****Layout Analysis & Table Recognition:****

"strengthened the ability of layout detection, table recognition, and formula recognition"

****Output Formats:****

The document mentions "convert results into Markdown files" but does not specify JSON output format explicitly.

****What it does:****

"PP-StructureV3 pipeline...has also added the ability to understand charts and restore reading order"

(2) Benchmark Comparisons

****vs. MinerU:****

On English documents, PP-StructureV3 achieved "Overall Edit? 0.145" vs MinerU-1.3.11 at "0.166"; for tables specifically, PP-StructureV3 "TableEdit? 0.159" vs MinerU "0.1613"

****vs. Other Systems:****

Marker-1.2.3 showed "0.336" overall vs PP-StructureV3 "0.145"; Mathpix achieved "0.191"

(3) Table-Structure Recognition

****Wired vs. Borderless:****

The document does not explicitly address handling of borderless/wireless vs. ruled tables. It only mentions test data "including elements such as tables, formulas, seals, and charts" without differentiating table types.

(4) Model Sizes & Hardware Requirements

****VRAM/GPU Support:****

On A100: "Average VRAM Usage (GB): 21.1" (Server PP-FormulaNet-L config); "Peak VRAM Usage (GB): 21.8"

On V100: "Average VRAM Usage (GB): 16.5"; "Peak VRAM Usage (GB): 17.0"

****Lightweight Config (A100):**** "Peak device memory (MB): 7456.00"

****GPU Support:****

Tested on "NVIDIA Tesla V100," "NVIDIA A100," "A10," "T4," and "H20"

(5) Python API / Integration

"For Python projects, you can directly integrate using the PaddleOCR Python API"

Multi-GPU: set device as "gpu:0,1,2,3" to enable parallel inference

(6) License Statement

****Not provided**** in this document excerpt.

(7) Version & Maintenance Status

Document references "Paddle 3.0," "PaddleOCR 3.0.0," and test environment "PaddleX @ develop" but provides no explicit release date or EOL status.

5. assistant (2026-05-31T04:18:04.477Z)

The WebFetch returned useful content but several fields were thin (license, exact VRAM tiers, table-type handling, version date). Let me get more precise verbatim numbers from the benchmark and config tables, and check the version/release info.

6. user (2026-05-31T04:18:13.904Z)

PP-StructureV3 Benchmark Data - Exact Numbers

(A) End-to-End Evaluation Table

****PP-StructureV3 scores:****

- Overall Edit: EN 0.145, ZH 0.206
- Table Edit: EN 0.159, ZH 0.109

****Comparison systems:****

- MinerU-0.9.3: Overall 0.15 (EN), 0.357 (ZH); Table 0.18 (EN), 0.344 (ZH)
- Marker-1.2.3: Overall 0.336 (EN), 0.556 (ZH); Table 0.619 (EN), 0.685 (ZH)
- Mathpix: Overall 0.191 (EN), 0.365 (ZH); Table 0.243 (EN), 0.32 (ZH)
- GOT-OCR: Overall 0.287 (EN), 0.411 (ZH); Table 0.459 (EN), 0.52 (ZH)
- Qwen2.5-VL-72B: Overall 0.214 (EN), 0.261 (ZH); Table 0.341 (EN), 0.262 (ZH)

(B) Inference Performance Benchmarks

****Configuration examples:****

- PP_StructureV3-default on Intel 8350C + A100: Peak VRAM 37370 MB, Avg VRAM 34165.68 MB, Avg latency 1.38s
- PP_StructureV3-lightweight on Intel 8350C + A100: Peak VRAM 7456 MB, Avg VRAM 7131.95 MB, Avg latency 0.61s

(C) Deployment/Acceleration Mentions

No sentences found mentioning ONNX, vLLM, or specific "high-performance inference" plugins in the provided content.

(D) VRAM Requirements

No explicit minimum or recommended VRAM specifications are stated in the document.

7. assistant (2026-05-31T04:18:20.390Z)

I now have solid verbatim benchmark numbers. I have enough to extract falsifiable claims. The key findings relevant to the muneem research question:

1. ****Lightweight config fits consumer GPUs**** (~7.4 GB peak VRAM) ? directly relevant to the 8-12 GB tier.
2. ****Table accuracy benchmarks**** ? PP-StructureV3 beats VLMs (Qwen2.5-VL-72B, GOT-OCR) and pipeline competitors (Marker, MinerU) on table edit distance, which directly informs the "is a VLM more reliable than OCR+table-structure?" question.
3. ****Python API integration**** ? confirms the Python path.
4. ****License**** ? notably absent from this page, which is a flag for the license-uncertainty concern.

Let me assess source quality: this is the official PaddlePaddle/PaddleOCR GitHub repository documentation ? primary source from the maintaining institution (Baidu). Now I'll record findings.

8. user (2026-05-31T04:18:30.640Z)

Structured output provided successfully

9. assistant (2026-05-31T04:18:40.759Z)

Done. I fetched and analyzed the PaddleOCR PP-StructureV3 algorithm documentation.

****Source quality: primary**** ? this is the official PaddlePaddle/PaddleOCR GitHub repository documentation, maintained by the model's creator (Baidu).

****Key findings for muneem (5 claims extracted):****

- ****Central / VRAM tier:**** The `PP\_StructureV3-lightweight` config peaks at ~7.4 GB VRAM (7456 MB) at ~0.61s/page ? fits comfortably in the 8-12 GB consumer GPU tier, the most attractive local option for muneem's hardware constraint.
- ****Central / accuracy vs VLMs:**** On table edit-distance, PP-StructureV3 (0.159 EN / 0.109 ZH) beats Qwen2.5-VL-72B (0.341/0.262), GOT-OCR (0.459/0.52), Marker (0.619/0.685), and MinerU

(0.18/0.344). This is direct evidence bearing on the research question of whether a deterministic OCR+table-structure pipeline is more reliable than a prompt-driven VLM for dense tables ? it favors the pipeline.

- **Central / Python integration:** Confirmed usable via the PaddleOCR Python API, Markdown output.

- **Supporting / heavy config caveat:** The `default` config needs ~37 GB peak VRAM ? outside the single consumer/prosumer GPU range, so the lightweight config is the one to target.

- **Supporting / license + table-type gap:** This page carries **no license statement** for code or weights, and does **not** distinguish borderless/wireless vs. ruled table handling. Both the commercial-license-safety check (critical per the research constraints) and the borderless-table reliability question must be verified from other PaddleOCR sources ? flagged as license uncertainty.

Two caveats on the benchmark interpretation, which the parent agent should weigh: these are Baidu's own published numbers (primary but not independent), and the table scores are on general-document corpora (OmniDocBench-style), not specifically Indian semi-ruled bank statements ? so they are indicative, not a guarantee of performance on muneem's target documents.